Numerical Methods
LAB 6

Ilona Urbaniak

May 20, 2026

1. EXTRA PROBLEM (not obligatory, just think all of us together):
   is there a way to speed the Newton's method in many dimensions using Aitken's acceleration? Try to speed up the "slow" example from the code.

   Solution: See the code: 1.ipynb

   and the paper at:
   `http://www.zen89632.zen.co.uk/R/EM_Aitken/em_aitken.pdf`

   Problem:
   use this code to speed up solving another slowly converging set of non-linear equations.

2. Solve (manually) using Gaussian substitution:

$$\begin{aligned} 4x_1 - x_2 + x_3 &= 8 \\ 2x_1 + 5x_2 + 2x_3 &= 3 \\ x_1 + 2x_2 + 4x_3 &= 11 \end{aligned}$$

$$\begin{aligned} 4x_1 - x_2 + 2x_3 &= 9 \\ 2x_1 + 4x_2 - x_3 &= -5 \\ x_1 + x_2 - 3x_3 &= -9 \end{aligned}$$

   In both cases the solution is: $x_1 = 1, \quad x_2 = -1, \quad x_3 = 3.$

   Solution for the first problem:

$$EQ_1 - 2EQ_2 \quad and \quad EQ_2 - 2EQ_3 \longrightarrow \begin{aligned} -11x_2 - 3x_3 &= 2 \quad (3) \\ 2x_2 + 5x_3 &= 13 \quad (4) \end{aligned}$$

next step:

$$\begin{aligned} 5.5EQ_4 + EQ_3 \longrightarrow 24.5x_3 &= 73.5 \\ \longrightarrow x_3 &= 3 \\ \longrightarrow x_2 &= -1 \\ \longrightarrow x_1 &= 1 \end{aligned}$$

3. This program:
   2.ipynb
   performs Gaussian elimination and pivoting. Pivoting means, that the equations are ordered in such a way, that the coefficients with the greatest absolute value are on the diagonal. The set of equations:

$$\begin{aligned} x_1 + 2x_2 + 4x_3 &= 11 \\ 2x_1 + 5x_2 + 2x_3 &= 3 \\ 4x_1 - x_2 + x_3 &= 8 \end{aligned}$$

   is replaced by:

$$\begin{aligned} 4x_1 - x_2 + x_3 &= 8 \\ 2x_1 + 5x_2 + 2x_3 &= 3 \\ x_1 + 2x_2 + 4x_3 &= 11 \end{aligned}$$

   .
   This is done by this part of code:

```
for k in range(n):
   for i in range(k,n):
      if abs(M[i][k]) > abs(M[k][k]):
         M[k], M[i] = M[i],M[k]
      else:
         pass
```

   Next the Gaussian elimination is done later by this part of code:

```
for k in range(n):
   for i in range(k,n):
# [....]

      for j in range(k+1,n):
         q = float(M[j][k]) / M[k][k]
         for m in range(k, n+1):
```

```
            M[j][m] -=  q * M[k][m]

x = [0 for i in range(n)]

x[n-1] =float(M[n-1][n])/M[n-1][n-1]
for i in range (n-1,-1,-1):
   z = 0
   for j in range(i+1,n):
      z = z  + float(M[i][j])*x[j]
   x[i] = float(M[i][n] - z)/M[i][i]

return x
```

Using this code solve the equation from the first exercise, compare to the library function linalg.solve from scipy:

```
x = scipy.linalg.solve(A,b)
```

4. This code:
   3.ipynb
   performs the LU decomposition together with pivoting. It returns three matrices: L (lower), U (upper) and P (pivoting matrix). You can solve a set of linear equations:

$$\mathbf{A}x = b$$

by first performing the LU decomposition:

$$\mathbf{A} = \mathbf{P} \cdot \mathbf{L} \cdot \mathbf{U}$$

then finding y from

$$\mathbf{L}y = b$$

and finally finding x from:

$$\mathbf{U}x = y$$

Please extend the code to solve a set of linear equations. Solve this set of equations (from lecture):

$$\begin{pmatrix} 1 & 2 & 1 \\ 1 & -2 & 2 \\ 2 & 12 & -2 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 4 \\ 4 \end{pmatrix} \tag{1}$$